

Uno Storage Chiave-Valore Replicato e Fortemente Consistente in Go con Protocollo di Consenso Raft-lite

Francesco Astolfi mat. 0357293

Corso di “Sistemi Distribuiti e Cloud Computing” A.A. 2025/26

Università degli Studi di Roma Tor Vergata

Docente: Prof.ssa Valeria Cardellini

<https://github.com/FrancescoAstolfiDev/sdcc.git>

Abstract: Questo lavoro presenta la progettazione e l'implementazione di uno storage chiave-valore distribuito, replicato e fortemente consistente in Go, basato su una variante semplificata dell'algoritmo di consenso Raft. Il sistema è organizzato in tre tipologie di nodo indipendenti (nodo di consenso, client proxy e servizi di snapshot/backup), coordinate tramite gRPC/Protocol Buffers internamente e un'interfaccia REST/JSON verso l'esterno, con Service Discovery e Circuit Breaker come pattern architetturali per la ricerca dinamica del leader e l'isolamento dei guasti. Oltre alle garanzie di sicurezza classiche di Raft, il sistema implementa il protocollo Read Index come requisito vincolante per le letture linearizzabili dai follower, e un meccanismo uniforme di catch-up basato su pull per la compattazione del log e il recupero dello stato. Il sistema è stato validato incrementalmente: da test unitari, a un trasporto in process, fino a container Docker reali su un'istanza Amazon EC2, e valutato sperimentalmente su due assi, la scalabilità al variare della dimensione del cluster (3/5/7 nodi) e la tolleranza al crash improvviso del leader. I risultati mostrano un andamento non monotono della latenza rispetto al numero di nodi, spiegabile dal bilanciamento tra lo scarico del carico di lettura sui follower e il costo crescente del quorum di scrittura, e un tempo di ripristino dopo un crash del leader dominato non dal costo dell'algoritmo di consenso (millisecondi) ma dal timeout configurato lato Client Proxy (secondi).

I. INTRODUZIONE

Il Progetto B5 chiede di progettare e implementare uno storage chiave-valore distribuito: replicato tra più nodi, fortemente consistente e senza alcuna dipendenza da uno storage centralizzato. Le repliche sono coordinate da un algoritmo di consenso implementato internamente. L'architettura è organizzata in tre tipologie di nodo, distribuibili in modo indipendente: nodo di consenso, client proxy e servizio di snapshot/backup. Comunicano tra loro tramite gRPC e Protocol Buffers, mentre verso l'esterno il sistema espone un'interfaccia REST/JSON. Sono stati implementati due pattern architetturali, Service Discovery e Circuit Breaker, per la ricerca dinamica del leader e l'isolamento dei guasti. La relazione include anche un'analisi sperimentale della scalabilità rispetto alla dimensione del cluster e della tolleranza a un crash improvviso del leader. Il sistema è containerizzato con Docker Compose e distribuito su una singola istanza Amazon EC2. Il codice sorgente completo del progetto, comprensivo di implementazione, test e script sperimentali, è disponibile pubblicamente nella repository GitHub indicata in intestazione. Questa relazione descrive l'architettura, le principali scelte progettuali con i relativi trade-off, l'approccio implementativo e di testing, e la metodologia di valutazione sperimentale.

II. ARCHITETTURA DEL SISTEMA

Il sistema è composto da quattro servizi indipendenti, ciascuno eseguito come container o binario Docker separato, più un client esterno che non contatta mai direttamente il cluster.

- **Nodo di consenso (stateful).** Un cluster di processi identici, che si distinguono solo per un campo role assegnato a runtime (Leader, Follower, Candidate). Insieme mantengono lo stato replicato della macchina a stati chiave-valore e implementano l'elezione del leader e la replicazione del log.
- **Client Proxy (stateless).** L'unico punto di ingresso esterno. Riceve richieste REST/JSON, individua il leader corrente del cluster tramite Service Discovery, inoltra la richiesta via gRPC e traduce la risposta di nuovo in REST/JSON. **Circuit Breaker.** Implementato all'interno del Client Proxy, con un'istanza di breaker per ogni nodo noto del cluster. Se i fallimenti verso un nodo si ripetono, il relativo breaker si apre: il traffico verso quel nodo si interrompe e la vista del cluster viene subito aggiornata tramite Service Discovery. Questo evita retry a cascata durante un'elezione in corso o un guasto di rete transitorio.
- **Servizio di Discovery.** Un registro leggero, sviluppato internamente, che interroga periodicamente l'endpoint GetStatus() (ruolo, termine, indirizzo) di ogni nodo di consenso ed espone una vista aggregata del cluster (leader corrente, follower attivi) al Client Proxy e al servizio di Snapshot & Backup. È tenuto deliberatamente fuori dal data path: non viene interrogato per ogni singola operazione chiave-valore, ma solo per ricerche di leader e membri, poco frequenti e cacheabili, così da non diventare un collo di bottiglia sulla latenza.
- **Servizio di Snapshot e Backup (stateless).** Un servizio asincrono in background che compatta periodicamente il log del leader in file di checkpoint durevoli, che qualsiasi nodo può poi usare per recuperare lo stato senza rigiocare l'intera storia.

Tutto il traffico inter-servizio e di coordinamento del consenso (RequestVote, AppendEntries, GetStatus, e ogni chiamata Proxy→Nodo) usa gRPC e Protocol Buffers; l'unico confine REST/JSON nel sistema è tra il client esterno e il Client Proxy. Questa separazione mantiene il contratto interno fortemente tipizzato ed efficiente, e offre allo stesso tempo un'interfaccia semplice e familiare ai chiamanti esterni.

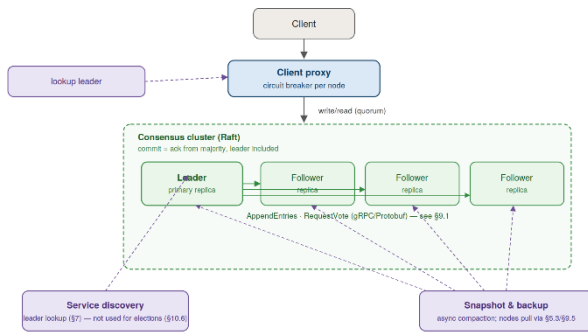


Fig. 1. Vista dinamica del sistema

III. PROGETTAZIONE DEL CONSENSO (RAFT-LITE)

Il livello di consenso implementa un sottoinsieme ridotto di Raft, focalizzato sulla correttezza: elezione del leader, replicazione del log con commit a quorum di maggioranza, regola di commit basata sul termine, e recovery da crash tramite persistenza su disco e snapshot. La dimensione del cluster è fissata al momento del deployment tramite Docker Compose; per gli esperimenti, però, il numero di nodi di consenso da attivare può comunque essere variato. Le proprietà di sicurezza garantite dall'implementazione sono le classiche garanzie di Raft: Election Safety (al massimo un leader per termine), Leader Append-Only, Log Matching, Leader Completeness e State Machine Safety. Una scrittura viene confermata (commit) solo dopo l'acknowledgment di una maggioranza dei nodi, leader incluso, e viene applicata alla macchina a stati locale solo dopo il commit, mai prima. Il leader mantiene nextIndex/matchIndex per ogni follower e usa i campi conflictIndex/conflictTerm restituiti per riparare il log di un follower in un solo round-trip dopo un AppendEntries rifiutato, invece di decrementare un indice alla volta. Lo stato di ogni nodo (termine corrente, voto espresso, log e macchina a stati chiave-valore) è persistito in modo indipendente su un volume Docker locale per nodo. La persistenza usa un file di log append-only con record prefissati dalla lunghezza, sincronizzato tramite flush/fsync prima di ogni acknowledgment di AppendEntries, più un piccolo file di metadata JSON per termine e voto, scritto con il pattern write-to-temp-then-rename. Al riavvio, un nodo ricostruisce la propria macchina a stati a partire dall'ultimo snapshot e rigioca la coda del log da quel punto in poi.

IV. PERCORSO DI LETTURA, SNAPSHOT E RECUPERO DELLO STATO

Per default, le letture sono servite esclusivamente dal leader. Come ottimizzazione, non come requisito di correttezza, il sistema punta anche al protocollo Read-Index per le letture dai follower: un follower conferma prima con il leader, tramite un round di heartbeat validato a quorum, qual è l'ultimo indice di commit sicuro da leggere; attende di averlo applicato localmente e solo allora risponde. In questo modo si evita traffico di lettura sul percorso CPU/IO del leader, senza sacrificare la consistenza. Poiché tutti i nodi girano su un'unica istanza EC2, la scelta non è motivata dalla prossimità di rete ma dallo scarico del carico dal leader quando c'è traffico concorrente di lettura e scrittura. La crescita del log è limitata da un meccanismo asincrono di recupero e compattazione basato su pull. Il servizio di Snapshot & Backup verifica periodicamente l'occupazione

del log sul leader e, superata una soglia configurabile, trasmette e archivia le entry committate più vecchie in un file di checkpoint contenente sia l'intervallo di log troncato sia il relativo snapshot della macchina a stati. Indipendentemente da questo, ogni nodo di consenso, non solo quelli che si uniscono al cluster, interroga periodicamente una piccola Snapshot Catalog API per verificare se esiste un checkpoint più recente di quello già applicato: in caso affermativo, lo scarica, lo adotta come nuova baseline e tronca in sicurezza il proprio log locale fino a quel punto. Questo unico meccanismo uniforme copre sia il caso di un nodo completamente nuovo che parte da stato vuoto, sia quello di un follower esistente rimasto semplicemente indietro. Le operazioni su una chiave vengono sempre valutate contro la macchina a stati in memoria, che rappresenta l'unica fonte di verità e viene mantenuta aggiornata sia dal replay del log sia dal caricamento degli snapshot; una Delete su una chiave non presente è un'operazione idempotente che non genera errore.

V. SEMANTICA DEL CLIENT

Il contratto REST esterno espone push (POST), update (PUT), get (GET) e delete (DELETE) su `/v1/kv/{key}`, mappati internamente sulle chiamate gRPC Put/Update/Get/Delete. La deduplicazione dei retry lato client (client-ID + numero di sequenza, come suggerito per workload Raft per rendere le query at most once) non è stata implementata deliberatamente, poiché le operazioni fondamentali dello store, SET e DELETE, sono naturalmente idempotenti: una entry duplicata causata da un retry del client dopo un timeout converge comunque allo stesso stato finale, indipendentemente da quante volte viene applicata.

VI. IMPLEMENTAZIONE E DEPLOYMENT

Il sistema è implementato in Go, con gRPC/Protocol Buffers per tutti i contratti RPC interni (consenso, servizio KV client-facing, discovery e API di trasferimento/catalogo degli snapshot), e con una libreria matura e open source per il circuit breaker, invece di un'implementazione scritta da zero, per concentrare lo sforzo implementativo sulla meccanica del consenso. Ognuna delle altre tipologie di nodo è costruita e distribuita come immagine Docker separata e orchestrata con Docker Compose; l'intero stack (un'istanza di Service Discovery, N nodi di consenso, una o più istanze di Client Proxy, e un'istanza di Snapshot & Backup) gira su una singola istanza Amazon EC2, con ogni nodo di consenso dotato di un volume persistente indipendente. Tutto il traffico interno resta su una rete bridge Docker privata; solo il Client Proxy è raggiungibile dall'esterno. Un problema implementativo rilevante, emerso e risolto durante lo sviluppo, riguarda la risoluzione DNS di default di gRPC. Il client gRPC di default usa lo scheme resolver dns, che cachea la risoluzione dell'hostname e non la aggiorna in modo affidabile dopo il riavvio di un peer con un nuovo IP sulla rete bridge Docker. La soluzione applicata dialoga ogni peer esplicitamente come passthrough://host:port, forzando una risoluzione fresca a ogni tentativo di riconnessione, combinata con waitForReady sulle chiamate che devono tollerare una breve finestra di riconnessione (AppendEntries) ma non su quelle sensibili alla latenza di elezione (RequestVote).

VII. TESTING E VALIDAZIONE

La correttezza è stata validata in modo incrementale. Prima i test unitari, sulla macchina a stati e sulle strutture dati del log in isolamento. Poi la logica di elezione e replicazione, verificata contro un trasporto fittizio in-process prima di introdurre qualsiasi rete reale. Infine, una volta collegato il sistema a vere chiamate gRPC tra container, test di integrazione mirati che coprono: la sicurezza del voto (un candidato con log obsoleto o più corto non vince mai contro un nodo più aggiornato), la sicurezza del commit attraverso crash ripetuti e forzati del leader, il recupero da crash del log durevole, il comportamento di redirect del proxy, l'apertura del circuit breaker su un follower ucciso, e la riduzione del log sotto carico di scrittura sostenuto tramite il percorso di snapshot/recupero. Una pipeline di integrazione continua (GitHub Actions) ha inoltre individuato una race condition durante lo sviluppo, oltre ad altri problemi risolti tramite debug live del cluster Docker Compose, ad esempio la risoluzione DNS di gRPC e le tempeste di split-vote all'avvio del cluster.

VIII. LIMITAZIONI

Un nodo isolato dalla maggioranza dei peer continua a incrementare il proprio termine a ogni timeout di elezione scaduto, senza mai poter vincere per costruzione, dato il suo log non aggiornato. Questo non viola alcuna proprietà di sicurezza, ma forza un leader legittimo a dimettersi non appena rientra in contatto con un termine artificialmente gonfiato, causando una rielezione superflua ma innocua. La soluzione standard in letteratura Raft è l'estensione Pre-Vote, non implementata in questo progetto per motivi di scope: il costo è solo di disponibilità (elezioni extra), mai di correttezza.

IX. METODOLOGIA DI VALUTAZIONE

Sono richiesti e sono stati eseguiti due scenari sperimentali contro il cluster distribuito su EC2. Scalabilità. Il numero di nodi di consenso è stato variato (es. 3, 5, 7) ed è stata misurata la latenza end-to-end delle scritture (Client Proxy → leader → AppendEntries → acknowledgment a quorum → risposta) con un piccolo generatore di carico in Go, registrando i percentili di latenza per ogni dimensione del cluster.

- p50 (mediana): la latenza "tipica", metà delle richieste sono più veloci, metà più lente. Rappresenta l'esperienza dell'utente medio.
- p95: il 95% delle richieste è più veloce di questo valore; solo il 5% è più lento. Cattura le richieste moderatamente sfortunate, come una AppendEntries che ha dovuto aspettare un follower lento o un breve jitter di rete.
- p99: solo l'1% delle richieste supera questo valore di attesa. Cattura la "coda" del comportamento: casi come un follower temporaneamente irraggiungibile, un round di replicazione che ha dovuto aspettare il quorum più lento, o un momento vicino a un'elezione.

Tolleranza ai guasti. Il leader corrente è stato ucciso bruscamente (docker stop/kill -9) e sono stati misurati tre tempi: tempo di rilevamento (scadenza dell'election timeout), tempo per eleggere un nuovo leader, e tempo fino

a quando il cluster converge e riprende correttamente a servire le richieste dei client.

X. RISULTATI

Scalabilità: misurata tramite una invocazione del docker compose separata in base al numero di nodi da mantenere nel cluster:

Test eseguiti dopo il warm-up del cluster e l'attesa dell'elezione del leader iniziale.

Nodi	Tipo	n	p50 (ms)	p95 (ms)	p99 (ms)
3	Read	689	67.42	133.52	166.75
3	Write	311	59.13	201.85	278.20
5	Read	673	39.05	63.99	77.53
5	Write	327	35.71	130.02	151.38
7	Read	687	65.13	119.67	153.11
7	Write	313	63.35	277.64	379.27

Non è una crescita monotona (3→5→7, sempre peggio) come ci si aspetterebbe da un modello puramente "più nodi = più costo di quorum": 5 nodi ha le latenze più basse di tutte e tre le configurazioni, sia in lettura che in scrittura, a ogni percentile.

- Read Index: con più nodi, ci sono più follower tra cui distribuire il 70% delle richieste di lettura in round robin; il leader stesso è meno sovraccarico. Questo migliora le prestazioni man mano che aggiungi nodi.
- Costo di quorum sulle scritture: più follower devono confermare un AppendEntries prima del commit; questo peggiora le prestazioni man mano che aggiungi nodi.

Fault Tolerance: Il sistema tollera il crash del leader e riprende a servire scritture in una media di 3.07s su 6 run, con una variabilità molto contenuta nonostante il tempo di elezione effettivo di Raft. Questo indica che il collo di bottiglia dominante nella recovery percepito dal client non è l'algoritmo di consenso, che si dimostra rapido e affidabile, ma il time-out configurato lato Client Proxy (PROXY_RPC_TIMEOUT_MS=2000ms), prima che una richiesta verso il leader caduto venga abbandonata e reindirizzata.

XI. CONCLUSIONI E LAVORI FUTURI

Il sistema realizzato soddisfa i requisiti funzionali e architetturali dell'assegnazione: tre tipologie di nodo indipendenti, comunicazione interna interamente su gRPC/Protocol Buffers, due pattern architetturali (Service Discovery, Circuit Breaker) esplicitamente identificabili nel codice, e un'implementazione del consenso scritta da zero piuttosto che delegata a una libreria matura, in linea con l'obiettivo di originalità del progetto. La valutazione sperimentale ha prodotto due risultati non banali, non anticipabili dalla sola teoria. Primo, la latenza non cresce in modo monotono con il numero di nodi: a 5 nodi il sistema è più veloce sia di 3 sia di 7, un effetto spiegabile dal bilanciamento tra lo scarico del carico di lettura sui follower abilitato dal Read-Index e il costo crescente del quorum di scrittura. Secondo, il tempo di ripristino percepito dal client dopo il crash del leader non è dominato dal costo dell'algoritmo di consenso (rapido e affidabile, nell'ordine di poche decine di millisecondi anche nel caso peggiore osservato), ma dal timeout fisso configurato lato Client Proxy: un collo di bottiglia architetturale distinto dal protocollo di consenso stesso. Il progetto presenta due

limitazioni accettate consapevolmente, entrambe motivate dallo scope e dai tempi di un progetto individuale: l'assenza dell'estensione Pre-Vote, che lascia un nodo isolato libero di generare rielezioni superflue (ma mai di violare la sicurezza) al rientro nel cluster; e la persistenza non sincrona di termine e voto su disco, che accetta un rischio teorico di doppio voto in caso di crash-restart nello stesso termine a fronte di uno sforzo implementativo sostenibile. Diverse direzioni di lavoro futuro emergono da questi risultati:

- Implementazione di Pre-Vote, per eliminare le rielezioni superflue causate da un nodo isolato che rientra nel cluster, chiudendo l'unica limitazione di liveness osservata.
- Validazione sperimentale della causa del collo di bottiglia nel recovery, riducendo `PROXY_RPC_TIMEOUT_MS` e verificando che il tempo di ripristino percepito dal client scenda proporzionalmente, a conferma diretta dell'ipotesi formulata in §9.
- Scaling orizzontale del solo Client Proxy (stateless per design) tramite un orchestratore con autoscaling basato sul carico, mantenendo invece fissa la membership del livello di consenso, coerentemente con la semantica di Raft, che richiede un protocollo di joint consensus (esplicitamente escluso dallo scope di questo progetto) per qualunque cambio dinamico dei nodi partecipanti.
- Deduplicazione dei retry lato client tramite ID cliente e numero di sequenza, per garantire semantica exactly-once anche per operazioni non naturalmente

idempotenti, qualora il modello di dati venisse esteso oltre le sole SET/DELETE.

- Autenticazione e cifratura del traffico (TLS su gRPC, autenticazione sul confine REST), assenti nell'implementazione attuale poiché fuori dallo scope dell'assegnazione ma necessarie per qualunque deployment reale oltre il contesto accademico.

XII. REFERENCES

- [1] D. Ongaro, J. Ousterhout. *"In Search of an Understandable Consensus Algorithm (Extended Version)."* Stanford University, 2014.
- [2] Francesco Astolfi. *"Progetto B5: Storage Chiave-Valore Replicato con Raft-lite."* GitHub repository, 2026.
Disponibile online:
<https://github.com/FrancescoAstolfiDev/sdcc.git>